Name	Vedant Dinkar Patil
UID no.	2024300170
Experiment No.	1

AIM:	**For the Water Jug Problem** you have 2 unmarked jugs(4L, 3L, Target = 2L), Each node is represented as (Jug1, Jug2).
PROBLEM STATEMENT :	**For the Water Jug Problem** you have 2 unmarked jugs(4L, 3L, Target = 2L), Each node is represented as (Jug1, Jug2). Allowed operations: Fill either jug to its maximum capacity Empty either jug completely Pour water from one jug to the other until one jug is full or the other is empty Show search tree how BFS and DFS explore the state space. Display Search tree, Traversal path for BFS and DFS. Give comparison of which method gives shortest path. Justify
Theory:	BFS gives shortest Path :- Think of BFS as taking things one step at a time. It explores every possible single move first, then checks all the two-move combinations, and so on. Because every action—whether it's pouring, filling, or emptying—counts as exactly one step, the very first time BFS hits the target, you know for a fact it's the most direct route. Depth-First Search (DFS), on the other hand, just picks a path and charges ahead until it hits a dead end or stumbles onto a solution. Because it doesn't care about how many steps it's taking, the first answer it finds might involve a bunch of completely unnecessary back-and-forth pouring before it finally gets it right. DFS will get you an answer, but it definitely doesn't guarantee the quickest way there.

PROGRAM:

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
using namespace std;

struct State {
   int jug1;
   int jug2;
};

// Function to print level-wise graph and find solution using BFS
void water_Jug_BFS(int cap1, int cap2, int target) {
   if (target > max(cap1, cap2) || target < 0) {
      cout << "Error!! Enter valid target value" << endl;
      return;
   }
   if (cap1 <= 0 || cap2 <= 0) {
      cout << "Error!! Enter valid jug capacities" << endl;
      return;
   }

   vector<vector<bool>> visited(cap1 + 1, vector<bool>(cap2 + 1, false));
   vector<vector<State>> parent(cap1 + 1, vector<State>(cap2 + 1, {-1, -1}));

   queue<State> q;
   State initialState = {0, 0};

   q.push(initialState);
   visited[0][0] = true;

   bool found = false;
   State targetState;

   cout << "\n--- Level-wise Graph Explorer (BFS) ---" << endl;
   int currentLevel = 0;

   while (!q.empty()) {
      int levelSize = q.size();
      cout << "Level " << currentLevel << " -> ";
```

```
    queue<State> levelQueue = q; // Copy to display nodes at this level
    while (!levelQueue.empty()) {
        State s = levelQueue.front();
        levelQueue.pop();
        cout << "(" << s.jug1 << ", " << s.jug2 << ") ";
    }
    cout << endl;

    // Process current level
    for (int i = 0; i < levelSize; i++) {
        State curr = q.front();
        q.pop();

        if (curr.jug1 == target || curr.jug2 == target) {
            targetState = curr;
            found = true;
            break;
        }

        vector<State> nextStates = {
            {cap1, curr.jug2},
            {curr.jug1, cap2},
            {0, curr.jug2},
            {curr.jug1, 0},
            {curr.jug1 - min(curr.jug1, cap2 - curr.jug2), curr.jug2 + min(curr.jug1, cap2 -
curr.jug2)},
            {curr.jug1 + min(curr.jug2, cap1 - curr.jug1), curr.jug2 - min(curr.jug2, cap1 -
curr.jug1)}
        };

        for (State next : nextStates) {
            if (!visited[next.jug1][next.jug2]) {
                visited[next.jug1][next.jug2] = true;
                parent[next.jug1][next.jug2] = curr;
                q.push(next);
            }
        }
    }
```

```
        if (found) break;
        currentLevel++;
    }

    if (!found) {
        cout << "\nThere is no solution for this target value" << endl;
        return;
    }

    // Reconstruct path
    vector<State> path;
    State curr = targetState;
    while (!(curr.jug1 == 0 && curr.jug2 == 0)) {
        path.push_back(curr);
        curr = parent[curr.jug1][curr.jug2];
    }
    path.push_back({0, 0});

    cout << "\nShortest Path to Target (" << target << ") using BFS:" << endl;
    for (int i = path.size() - 1; i >= 0; i--) {
        cout << "(" << path[i].jug1 << ", " << path[i].jug2 << ")";
        if (i > 0) cout << " -> ";
    }
    cout << "\n";
}

// Function to find solution using DFS
void water_Jug_DFS(int cap1, int cap2, int target) {
    if (target > max(cap1, cap2) || target < 0) {
        cout << "Error!! Enter valid target value" << endl;
        return;
    }
    if (cap1 <= 0 || cap2 <= 0) {
        cout << "Error!! Enter valid jug capacities" << endl;
        return;
    }

    vector<vector<bool>> visited(cap1 + 1, vector<bool>(cap2 + 1, false));
    vector<vector<State>> parent(cap1 + 1, vector<State>(cap2 + 1, {-1, -1}));
```

```
    stack<State> st;
    State initialState = {0, 0};

    st.push(initialState);
    visited[0][0] = true;

    bool found = false;
    State targetState;

    cout << "\n--- Exploring graph using DFS ---" << endl;

    while (!st.empty()) {
        State curr = st.top();
        st.pop();

        if (curr.jug1 == target || curr.jug2 == target) {
            targetState = curr;
            found = true;
            break;
        }

        vector<State> nextStates = {
            {cap1, curr.jug2},
            {curr.jug1, cap2},
            {0, curr.jug2},
            {curr.jug1, 0},
            {curr.jug1 - min(curr.jug1, cap2 - curr.jug2), curr.jug2 + min(curr.jug1, cap2 -
curr.jug2)},
            {curr.jug1 + min(curr.jug2, cap1 - curr.jug1), curr.jug2 - min(curr.jug2, cap1 -
curr.jug1)}
        };

        // Push in reverse order so that the first element in nextStates is processed first (LIFO
correction)
        for (int i = nextStates.size() - 1; i >= 0; i--) {
            State next = nextStates[i];
            if (!visited[next.jug1][next.jug2]) {
                visited[next.jug1][next.jug2] = true;
                parent[next.jug1][next.jug2] = curr;
                st.push(next);
```

```
                cout << "(" << curr.jug1 << ", " << curr.jug2 << ") -> ("
                    << next.jug1 << ", " << next.jug2 << ")" << endl;
            }
        }
    }

    if (!found) {
        cout << "\nThere is no solution for this target value" << endl;
        return;
    }

    // Reconstruct path
    vector<State> path;
    State curr = targetState;
    while (!(curr.jug1 == 0 && curr.jug2 == 0)) {
        path.push_back(curr);
        curr = parent[curr.jug1][curr.jug2];
    }
    path.push_back({0, 0});

    cout << "\nPath to Target (" << target << ") using DFS:" << endl;
    for (int i = path.size() - 1; i >= 0; i--) {
        cout << "(" << path[i].jug1 << ", " << path[i].jug2 << ")";
        if (i > 0) cout << " -> ";
    }
    cout << "\n";
}

int main() {
    int cap1, cap2, target;

    cout << "Enter capacity of Jug 1: ";
    cin >> cap1;
    cout << "Enter capacity of Jug 2: ";
    cin >> cap2;
    cout << "Enter target volume: ";
    cin >> target;

    int choice;
    char repeat;
```

```
    do {
        cout << "\nChoose the method to solve:" << endl;
        cout << "1. BFS (Level-wise Graph)" << endl;
        cout << "2. DFS" << endl;
        cout << "Enter your choice (1 or 2): ";
        cin >> choice;

        if (choice == 1) {
            water_Jug_BFS(cap1, cap2, target);
        } else if (choice == 2) {
            water_Jug_DFS(cap1, cap2, target);
        } else {
            cout << "Invalid choice!" << endl;
        }

        cout << "\nWould you like to try the other method? (y/n): ";
        cin >> repeat;
    } while (repeat == 'y' || repeat == 'Y');

    return 0;
}
```

Output:

```
Enter capacity of Jug 1: 4
Enter capacity of Jug 2: 3
Enter target volume: 2

Choose the method to solve:
1. BFS (Level-wise Graph)
2. DFS
Enter your choice (1 or 2): 1

--- Level-wise Graph Explorer (BFS) ---
Level 0 -> (0, 0)
Level 1 -> (4, 0) (0, 3)
Level 2 -> (4, 3) (1, 3) (3, 0)
Level 3 -> (1, 0) (3, 3)
Level 4 -> (0, 1) (4, 2)

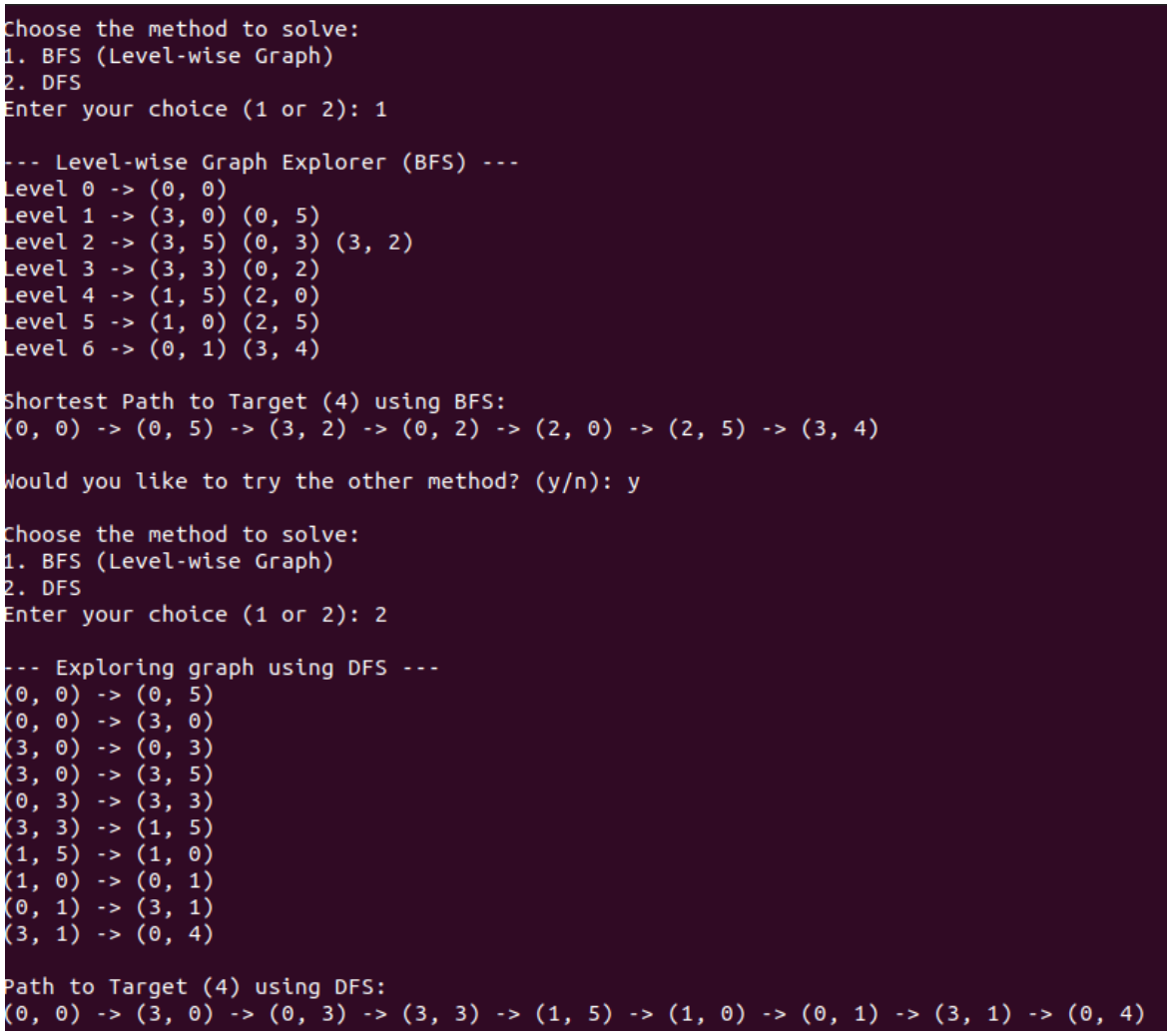
Shortest Path to Target (2) using BFS:
(0, 0) -> (0, 3) -> (3, 0) -> (3, 3) -> (4, 2)

Would you like to try the other method? (y/n): y

Choose the method to solve:
1. BFS (Level-wise Graph)
2. DFS
Enter your choice (1 or 2): 2

--- Exploring graph using DFS ---
(0, 0) -> (0, 3)
(0, 0) -> (4, 0)
(4, 0) -> (1, 3)
(4, 0) -> (4, 3)
(1, 3) -> (1, 0)
(1, 0) -> (0, 1)
(0, 1) -> (4, 1)
(4, 1) -> (2, 3)

Path to Target (2) using DFS:
(0, 0) -> (4, 0) -> (1, 3) -> (1, 0) -> (0, 1) -> (4, 1) -> (2, 3)
```

	
CONCLUSION:	To wrap things up, this classic puzzle perfectly highlights how differently these two algorithms "think." Both BFS and DFS will eventually figure out how to measure exactly 2 liters, but their approaches are completely different. If you just want *an* answer and don't really care how many times you have to pointlessly fill and dump the water, DFS will eventually stumble across a solution. But if you want the smartest, most efficient way to solve the puzzle with the fewest moves possible, BFS is exactly what you need. It naturally maps out the shortest, most direct route—saving you a lot of unnecessary pouring!